Deep Guide to Notebooks 05–09

Hateful Meme Detection Project

From preprocessing to AUROC optimisation

Project:	hateful-meme-project
Scope:	notebooks 05_preprocessing through 09_auroc-optimization
Focus:	data pipeline, baselines, architecture, training, fine-tuning
Prepared:	April 13, 2026

This document is written for a reader starting from scratch. It explains the conceptual background, the modelling prerequisites, the code walkthrough, and then the purpose and internal logic of notebooks 05, 06, 07, 08, and 09 in detail.

Contents

1 Where Notebooks 05–09 Fit in the Pipeline **3**
1.1 What Changed Between Notebook 04 and 05 . . . 3
1.2 Modelling Prerequisites for These Notebooks . . . 4

2 Notebook 05: Preprocessing **5**
2.1 High-Level Goal . . . 5
2.2 Cell Group 1: Environment and Data Loading . . . 5
2.3 Cell Group 2: Image Path Resolution . . . 5
2.4 Cell Group 3: Text Cleaning . . . 6
2.5 Cell Group 4: Image Preprocessing Pipelines . . . 6
2.5.1 Training Transform (with augmentation) . . . 6
2.5.2 Evaluation Transform (deterministic) . . . 6
2.6 Cell Group 5: Augmentation Visualisation . . . 7
2.7 Cell Group 6: MemeDataset Class . . . 7
2.8 Cell Group 7: Label Balancing . . . 7
2.9 Cell Group 8: Save Cleaned Splits . . . 8
2.10 What Notebook 05 Teaches . . . 8

3 Notebook 06: Baselines **9**
3.1 High-Level Goal . . . 9
3.2 Baseline 1: TF-IDF + Logistic Regression (Text Only) . . . 9
3.3 Baseline 2: RoBERTa Text Classifier . . . 9
3.4 Baseline 3: CLIP Image-Only Linear Probe . . . 10
3.5 Baseline 4: CLIP Simple Concatenation Fusion . . . 10
3.6 Baseline Summary Table . . . 10
3.7 What Notebook 06 Teaches . . . 11

4 Notebook 07: Main Model Architecture **12**
4.1 High-Level Goal . . . 12
4.2 Component 1: CLIPEncoder . . . 12
4.3 Component 2: CrossAttentionFusion . . . 13
4.4 Component 3: ClassificationHead . . . 13
4.5 Architecture Diagram . . . 13
4.6 Parameter Count . . . 14
4.7 Forward Pass Smoke Test . . . 14
4.8 Cross-Attention Visualisation . . . 15
4.9 What Notebook 07 Teaches . . . 15

5 Notebook 08: Training **16**
5.1 High-Level Goal . . . 16
5.2 Training Configuration . . . 16
5.3 Weighted Focal Loss . . . 17

5.4 Learning Rate Schedule . . . 17
5.5 Training Loop Details . . . 17
5.6 Ablation Results . . . 18
5.7 Training Curves Plot . . . 18
5.8 What Notebook 08 Teaches . . . 18

6 Notebook 09: AUROC Optimisation (Phase 2) 19
6.1 High-Level Goal . . . 19
6.2 Why Phase 2 Is Separate . . . 19
6.3 Configuration Differences from Notebook 08 . . . 19
6.4 Text Augmentation Strategy . . . 20
6.5 Focal Loss with Label Smoothing . . . 20
6.6 Selective Backbone Unfreezing . . . 21
6.7 Training Output and Results . . . 21
6.8 Why AUROC Is the Optimisation Target . . . 22
6.9 What Notebook 09 Teaches . . . 22

7 Cross-Notebook Patterns and Design Lessons 23
7.1 Progressive Complexity . . . 23
7.2 Each Notebook Is Self-Contained . . . 23
7.3 AUROC as the North Star Metric . . . 23
7.4 Ablation-Driven Design . . . 23
7.5 Regularisation Philosophy . . . 24

8 Concise Notebook-by-Notebook Summary 25

9 Final Interpretation 26

A Key Numbers Used in This Report 27

B Suggested Reading Order for a Beginner 28

Chapter 1

Where Notebooks 05–09 Fit in the Pipeline

Notebooks 00 through 04 covered the foundational stages of the hateful meme detection project: environment verification, problem scoping, data inspection, exploratory data analysis, and OCR quality auditing. By the end of notebook 04, the project had established:

- A reliable, portable environment setup.
- A clearly defined binary classification task: $f(x_{\text{image}}, x_{\text{text}}) \to y \in \{0, 1\}$.
- A verified dataset with 8,500 training, 500 dev, and 1,000 test samples.
- Confidence in the OCR text quality and a fallback policy.

Notebooks 05–09 move from **understanding data** to **building and optimising models**. The five notebooks covered here are:

1. `05_preprocessing.ipynb`: text cleaning, image transforms, dataset class, class balancing, and saving cleaned splits.
2. `06_baselines.ipynb`: four progressively stronger baselines (TF-IDF, RoBERTa, CLIP-image, CLIP-fusion) to establish performance floors.
3. `07-main-model.ipynb`: full architecture definition—frozen CLIP encoder, bidirectional cross-attention fusion, MLP head.
4. `08-training.ipynb`: end-to-end training loop with focal loss, cosine scheduling, mixed precision, and four ablation variants.
5. `09-auroc-optimization.ipynb`: phase 2 fine-tuning with partial backbone unfreezing, label smoothing, and text augmentation.

1.1 What Changed Between Notebook 04 and 05

Notebook 04 ended with an engineering decision: the dataset's human-cleaned `text` field is reliable enough to use as the primary text source, with EasyOCR as a fallback for short or empty entries. Notebook 05 acts on that decision by building the full preprocessing pipeline that downstream training notebooks consume.

The transition is deliberate. The project does not jump from raw data to a model. It inserts a *reproducible preprocessing stage* that normalises text, defines image transforms, and produces cleaned JSONL files. This separation ensures that every later notebook trains on identical data.

1.2 Modelling Prerequisites for These Notebooks

The reader should be familiar with a few additional concepts beyond what notebooks 00–04 required:

- **Transfer learning**: using a pretrained model (CLIP) as a feature extractor and training only a small head on top.
- **CLIP (Contrastive Language-Image Pre-training)**: a model trained by OpenAI to align images and text in a shared 512-dimensional embedding space.
- **Cross-attention**: a transformer mechanism where one modality queries the other to produce interaction-aware representations.
- **Focal loss**: a loss function that down-weights easy examples and focuses training on hard, misclassified cases.
- **AUROC**: area under the receiver operating characteristic curve, a threshold-independent ranking metric.
- **Mixed-precision training**: using `float16` for forward passes to reduce GPU memory and increase throughput.

Chapter 2

Notebook 05: Preprocessing

Notebook 05 is the bridge between exploratory analysis and model training. Its purpose is to transform raw dataset fields into a uniform, model-ready format. Every subsequent notebook depends on the artefacts produced here.

2.1 High-Level Goal

The notebook accomplishes five tasks:

1. Clean and normalise meme text.
2. Define CLIP-compatible image transforms (training and evaluation).
3. Visualise augmentation effects.
4. Build a reusable `MemeDataset` PyTorch class.
5. Compute class-balancing weights and save cleaned splits.

2.2 Cell Group 1: Environment and Data Loading

The first cells replicate the data directory discovery logic from earlier notebooks. This is intentional: each notebook is designed to be self-contained. The code searches for JSONL files and an image directory under `/kaggle/input`, `./data`, or an environment variable override.

Output:

```
Using DATA_DIR : /kaggle/input/.../data
Using IMG_DIR  : /kaggle/input/.../data/img
Train=8500, Dev=500, Test=1000
```

This confirms the same dataset partitions used throughout the project.

2.3 Cell Group 2: Image Path Resolution

The `resolve_image_path` function handles the fact that the `img` field in the JSONL may store paths like `img/12345.png` or just `12345.png`. The function tries multiple candidate paths:

- Absolute path (if the reference is absolute).
- `data_dir / image_ref` (direct join).

- `data_dir / "img" / filename` (strip the prefix).

This robustness is important because the same notebook runs on Kaggle, locally on Windows, and in CI. A fragile hard-coded path would break portability.

2.4 Cell Group 3: Text Cleaning

The `clean_text` function applies a deliberate sequence of normalisations:

1. Fix encoding errors via UTF-8 round-trip.
2. Remove URLs (`https?://...`).
3. Remove @-mentions.
4. Expand hashtags: `#hate` → `hate`.
5. Collapse multiple whitespace into a single space.
6. Strip leading and trailing whitespace.

The key design choice is what the function does *not* do: it preserves case ("DISGUSTING" stays uppercase), preserves punctuation that carries sentiment (exclamation marks), and does not stem or lemmatise. The reasoning is that CLIP's text encoder was trained on naturally cased internet text, so aggressive normalisation would destroy signal.

Output example:

```
Before: 'CHECK THIS OUT https://example.com #hate @someone DISGUSTING!'
After : 'CHECK THIS OUT hate DISGUSTING!'
```

Empty or invalid text fields are replaced with `[no text]` as a sentinel, ensuring no downstream `NaN` crashes.

2.5 Cell Group 4: Image Preprocessing Pipelines

Two transform pipelines are defined:

2.5.1 Training Transform (with augmentation)

```
train_transform = transforms.Compose([
    transforms.Resize((256, 256)),
    transforms.RandomCrop(224),
    transforms.RandomHorizontalFlip(p=0.3),
    transforms.ColorJitter(brightness=0.2, contrast=0.2,
                           saturation=0.1),
    transforms.ToTensor(),
    transforms.Normalize(CLIP_MEAN, CLIP_STD),
])
```

2.5.2 Evaluation Transform (deterministic)

```
eval_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(CLIP_MEAN, CLIP_STD),
])
```

The mean and standard deviation values $\mu = [0.481, 0.458, 0.408]$ and $\sigma = [0.269, 0.261, 0.276]$ are CLIP's pretraining statistics. Using different normalisation would break the pretrained feature extractor.

Why augmentation is mild: memes are photographs with overlaid text. Aggressive spatial transforms (large rotations, perspective warps) would distort the text layout and change the meaning. The augmentations chosen here —random crop, mild flip, colour jitter—preserve semantic content while adding training diversity.

2.6 Cell Group 5: Augmentation Visualisation

The notebook renders the original image alongside five augmented versions. This is not decorative. It serves as a visual sanity check: the reader can confirm that augmented images still look like recognisable memes with readable text.

2.7 Cell Group 6: MemeDataset Class

The `MemeDataset` is a PyTorch `Dataset` subclass that:

1. Loads an image from disk using the robust path resolver.
2. Reads the `clean_text` field (falling back to raw `text`).
3. Passes both through the CLIP processor, which handles tokenisation and image resizing.
4. Returns a dictionary with keys `pixel_values`, `input_ids`, `attention_mask`, and `label`.

Output of a smoke test:

```
Sample batch keys: ['pixel_values', 'input_ids',
                    'attention_mask', 'label']
  pixel_values shape : torch.Size([3, 224, 224])
  input_ids shape    : torch.Size([77])
  attention_mask shape: torch.Size([77])
  label              : 0
```

The `input_ids` length of 77 matches CLIP's maximum context window. Texts shorter than 77 tokens are padded; longer texts are truncated. This is a constraint of the CLIP tokeniser, not a project-specific choice.

2.8 Cell Group 7: Label Balancing

The training set has 5,450 non-hateful (64.1%) and 3,050 hateful (35.9%) samples. Without correction, a model could achieve decent accuracy by predicting "non-hateful" more often.

The notebook computes inverse-frequency class weights:

$$w_c = \frac{N}{2 \cdot N_c}$$

```
Class weights: non-hateful=0.780, hateful=1.393
```

A `WeightedRandomSampler` is then created, which oversamples hateful memes during training so that each batch is approximately 50/50. This is preferable to class-weighted loss alone because it also balances gradient contribution per batch.

2.9 Cell Group 8: Save Cleaned Splits

The notebook writes three JSONL files to the output directory:

```
Saved train_clean.jsonl  (8500 rows)
Saved dev_clean.jsonl    (500 rows)
Saved test_clean.jsonl   (1000 rows)
```

Each file contains the columns `id`, `img`, `label`, `text`, and `clean_text`. Downstream notebooks load these files instead of the raw dataset, ensuring consistent preprocessing.

2.10 What Notebook 05 Teaches

This notebook teaches the difference between **having raw data** and **having model-ready data**. In a proper ML pipeline, preprocessing is a distinct, reproducible stage. It is not embedded inside the training loop where it would be invisible, untestable, and impossible to audit.

The design decisions—mild augmentation, CLIP-specific normalisation, sentinel text for empty fields, balanced sampling—are all deliberate trade-offs documented inline. None of them would be obvious to a reader who skipped this notebook.

Chapter 3

Notebook 06: Baselines

Before building a complex multimodal model, the project establishes performance floors through four progressively stronger baselines. This is a fundamental scientific practice: without baselines, there is no way to know whether a sophisticated model actually adds value.

3.1 High-Level Goal

The notebook trains four models and compares them on the dev set using accuracy, F1 (hateful class), macro F1, and AUROC. Each baseline isolates a different modality or fusion strategy.

3.2 Baseline 1: TF-IDF + Logistic Regression (Text Only)

This is the simplest reasonable text classifier. It uses:

- TF-IDF vectorisation with up to 20,000 features, unigrams and bigrams, sublinear TF scaling, and a minimum document frequency of 2.
- Logistic regression with balanced class weights and L2 regularisation ($C = 1.0$).

```
tfidf_pipe = Pipeline([
    ("tfidf", TfidfVectorizer(max_features=20000,
              ngram_range=(1, 2), sublinear_tf=True)),
    ("clf",   LogisticRegression(C=1.0, max_iter=1000,
              class_weight="balanced")),
])
```

Results:

```
1. TF-IDF + LR (text)  Acc=0.560  F1(hate)=0.463
                        MacroF1=0.545  AUROC=0.594
```

Interpretation: the AUROC of 0.594 confirms what the project hypothesised in notebook 01: text alone is insufficient because hateful meaning often emerges from the *combination* of image and text. However, the baseline is above random (0.50), indicating that some hateful memes do contain explicit textual cues.

3.3 Baseline 2: RoBERTa Text Classifier

The second baseline replaces TF-IDF with a pretrained transformer encoder (`roberta-base`). The model is fine-tuned for 3 epochs with a small learning rate.

Results:

```
2. RoBERTa (text only)  Acc=0.548  F1(hate)=0.331
                         MacroF1=0.495  AUROC=0.645
```

Interpretation: RoBERTa achieves a higher AUROC (0.645 vs 0.594) but lower accuracy and F1 than TF-IDF. This means the ranking quality is better—RoBERTa assigns higher scores to hateful memes on average—but the default 0.5 threshold produces worse binary predictions.

The gap between AUROC and accuracy is a recurring theme in this project. It is why AUROC was chosen as the primary metric in notebook 01: it captures ranking quality independent of threshold choice.

3.4 Baseline 3: CLIP Image-Only Linear Probe

This baseline uses CLIP's vision encoder to extract 512-dimensional image features and trains a simple linear classifier on top. CLIP weights are frozen; only the linear head is trainable.

Results:

```
3. CLIP image + linear (img)  Acc=0.564  F1(hate)=0.466
                               MacroF1=0.549  AUROC=0.647
```

Interpretation: the image-only AUROC (0.647) matches RoBERTa's text-only AUROC. This is a significant finding: *neither modality alone is meaningfully better than the other* on this dataset. The confounder-style construction of Meta Hateful Memes ensures that both text and image carry partial but incomplete signal.

3.5 Baseline 4: CLIP Simple Concatenation Fusion

The fourth baseline extracts both text and image embeddings from CLIP, concatenates them into a 1024-dimensional vector, and trains a small MLP on top. This is the simplest possible multimodal approach.

Results:

```
4. CLIP simple concat fusion  Acc=0.634  F1(hate)=0.583
                                MacroF1=0.628  AUROC=0.700
```

Interpretation: the jump from ∼0.647 (unimodal) to 0.700 (fusion) demonstrates that combining modalities adds genuine value, even with naive concatenation. This validates the project's core architectural bet: a multimodal approach is necessary.

3.6 Baseline Summary Table

Baseline	Acc	F1(hate)	MacroF1	AUROC
1. TF-IDF + LR (text)	0.560	0.463	0.545	0.594
2. RoBERTa (text only)	0.548	0.331	0.495	0.645
3. CLIP image + linear (img)	0.564	0.466	0.549	0.647
4. CLIP concat fusion	0.634	0.583	0.628	0.700

Table 3.1: Dev-set performance of the four baselines.

The table reveals a clear progression:

- Single-modality baselines hover around AUROC 0.59–0.65.
- Even naive fusion (concatenation) reaches 0.70.
- The gap motivates investing in a more sophisticated fusion mechanism (cross-attention), which is the subject of notebook 07.

3.7 What Notebook 06 Teaches

This notebook teaches **experimental methodology**: always establish baselines before building complex models. The four baselines form a ladder of increasing complexity. Each rung answers a specific question:

1. Can simple text features classify hateful memes? (Partially.)
2. Does a stronger text encoder help? (Ranking improves, but not enough.)
3. Does image information alone match text? (Roughly, yes.)
4. Does combining modalities help, even naively? (Yes, substantially.)

Without this ladder, the later cross-attention model would have no context. The reader would not know whether AUROC 0.72 is good or just slightly above random.

Chapter 4

Notebook 07: Main Model Architecture

Notebook 07 defines the full multimodal classifier that the project uses for all subsequent training, evaluation, and explainability experiments. It contains no training—only architecture definition, parameter counting, and a forward-pass smoke test.

4.1 High-Level Goal

The notebook defines three modular components:

1. `CLIPEncoder`: frozen CLIP ViT-B/32 backbone.
2. `CrossAttentionFusion`: bidirectional cross-attention between image and text embeddings.
3. `ClassificationHead`: MLP with GELU activations and dropout.

These are composed into `HatefulMemeClassifier`, a single `nn.Module` that takes raw pixel values and token IDs and returns logits plus attention weights.

4.2 Component 1: CLIPEncoder

```
class CLIPEncoder(nn.Module):
    def __init__(self, model_name="openai/clip-vit-base-patch32"):
        super().__init__()
        self.clip = CLIPModel.from_pretrained(model_name)
        for param in self.clip.parameters():
            param.requires_grad_(False)

    def forward(self, pixel_values, input_ids, attention_mask):
        img_emb = F.normalize(self.clip.get_image_features(
            pixel_values=pixel_values), dim=-1)
        txt_emb = F.normalize(self.clip.get_text_features(
            input_ids=input_ids,
            attention_mask=attention_mask), dim=-1)
        return img_emb, txt_emb
```

Why freeze? CLIP was pretrained on 400 million image-text pairs. The hateful meme dataset has only 8,500 training samples. Fine-tuning all 151M parameters on this small dataset would

cause catastrophic forgetting: the encoder would overfit to memes and lose its general visual-linguistic understanding.

Why L2-normalise? CLIP was trained with a cosine-similarity objective. Normalising the embeddings keeps them on the unit hypersphere, which is the geometry the model expects. Without normalisation, downstream layers would need to learn magnitude scaling, wasting capacity.

4.3 Component 2: CrossAttentionFusion

The fusion module uses bidirectional cross-attention:

- **Image → Text**: the image embedding queries the text embedding. This asks "what part of the text is relevant given this image?"
- **Text → Image**: the text embedding queries the image embedding. This asks "what part of the image is relevant given this text?"

Each direction uses multi-head attention with 4 heads and residual connections, followed by layer normalisation. The two attended outputs are concatenated to produce a 1024-dimensional fused representation.

Why cross-attention over concatenation? Simple concatenation treats image and text features as independent. Cross-attention allows the model to discover *which* image features interact with *which* text features. For hateful memes, this interaction is the core signal: a picture of a person is not hateful, and the text "look who's coming to dinner" is not hateful, but together they can be.

Why bidirectional? Unidirectional attention (e.g., only image querying text) would miss cases where the text modifies the image's meaning. Bidirectional attention captures interactions in both directions.

4.4 Component 3: ClassificationHead

The MLP head maps the 1024-dimensional fused vector to two logits:

```
MLP Head: 1024 -> 256 -> 128 -> 2
Activations: GELU
Dropout: 0.3
```

Why GELU over ReLU? GELU (Gaussian Error Linear Unit) provides smoother gradients than ReLU and is the standard activation in transformer architectures. Using GELU maintains consistency with CLIP's internal activations.

Why dropout 0.3? With only ∼2.4M trainable parameters and 8,500 training samples, overfitting is a serious risk. A 30% dropout rate is aggressive but appropriate for this data-to-parameter ratio.

4.5 Architecture Diagram

The notebook displays this ASCII architecture diagram:

```
  +----------------+    +----------------+
  |  Meme Image    |    |  OCR Text      |
  +-------+--------+    +-------+--------+
```

```
          |                       |
          v                       v
  +-------------------------------------+
  |  CLIP ViT-B/32  (FROZEN)            |
  |  Vision Encoder | Text Encoder      |
  +-------+-----------------+-----------+
  img_emb | (B,512)  txt_emb | (B,512)
          |                  |
          v                  v
  +-------------------------------------+
  |  Bidirectional Cross-Attention      |
  |  img->txt + txt->img + residual     |
  |  LayerNorm x2                       |
  +----------------+--------------------+
                   | fused (B,1024)
                   v
  +-------------------------------------+
  |  MLP Head: 1024->256->128->2        |
  |  GELU activations, Dropout 0.3     |
  +----------------+--------------------+
                   |
                   v
         logits (B, 2) -> softmax -> P(hateful)
```

4.6 Parameter Count

```
Total parameters      : 153,676,163
Frozen (CLIP)         : 151,277,313  (98.4%)
Trainable             : 2,398,850   (1.6%)
```

The trainable fraction is 1.6%. This is efficient: the model leverages CLIP's 151M pretrained parameters while only optimising 2.4M parameters for the hateful-meme task.

Per-module breakdown:

- `CLIPEncoder`: ∼151.3M (all frozen).
- `CrossAttentionFusion`: ∼2.1M (trainable).
- `ClassificationHead`: ∼0.3M (trainable).

4.7 Forward Pass Smoke Test

The notebook creates a dummy image (solid colour) and dummy text, passes them through the model, and verifies output shapes:

```
Logits       : [[0.167, 0.098]]
Probabilities: [non-hateful=0.517, hateful=0.483]
img_attn shape: torch.Size([1, 1, 1])
txt_attn shape: torch.Size([1, 1, 1])

Forward pass OK.
```

The near-uniform probabilities (0.52 vs 0.48) are expected for an untrained model with random head weights. This confirms the architecture produces valid outputs without crashes.

4.8 Cross-Attention Visualisation

The notebook plots attention weight heatmaps for both directions on the dummy input. With pool-level embeddings (single vectors per modality), the attention matrices are 1×1. The visualisation becomes meaningful in notebook 11 (Explainability) when token-level features are used.

4.9 What Notebook 07 Teaches

This notebook teaches **architecture-as-documentation**. By isolating the model definition in its own notebook—separate from training and evaluation—the project makes the design choices inspectable.

Every class is defined inline with no external source imports. A reader can understand the full model by reading one notebook. The forward-pass test proves the model compiles and runs before investing GPU hours in training.

Chapter 5

Notebook 08: Training

Notebook 08 is the first time the project actually trains the multimodal model. It implements the full training loop: data loading, loss computation, gradient updates, learning rate scheduling, validation, and checkpoint saving.

5.1 High-Level Goal

The notebook trains **four model variants** and compares them:

1. **cross_attention**: the full model from notebook 07.
2. **image_only**: only CLIP image features, no text.
3. **text_only**: only CLIP text features, no image.
4. **late_fusion**: independent text and image classifiers whose logits are averaged (no cross-attention).

This ablation structure is a direct scientific validation of the cross-attention design choice.

5.2 Training Configuration

The training hyperparameters are collected in a `CFG` dictionary:

```
CFG = {
    'clip_model'   : 'openai/clip-vit-base-patch32',
    'batch_size'   : 32,
    'epochs'       : 10,
    'lr'           : 2e-4,
    'weight_decay' : 0.01,
    'warmup_steps' : 100,
    'focal_gamma'  : 2.0,
    'class_weights': [0.78, 1.39],
    'max_text_len' : 77,
}
```

Design rationale:

- **Learning rate** 2×10^{-4}: aggressive for a head-only training (CLIP is frozen), but appropriate because the cross-attention and MLP layers are randomly initialised.
- **Batch size 32**: fits comfortably in a T4 GPU's 16 GB VRAM with mixed precision.

- **Weight decay 0.01**: standard AdamW regularisation to prevent overfitting.
- **Warmup 100 steps**: prevents large gradient updates in the first few batches when the head is randomly initialised.

5.3 Weighted Focal Loss

The loss function addresses two problems simultaneously:

1. **Class imbalance**: the `class_weights` parameter up-weights the hateful class during loss computation.
2. **Hard-example mining**: the focal term $(1 - p_t)^\gamma$ with $\gamma = 2.0$ down-weights confidently correct predictions and focuses the gradient on difficult cases.

```
class WeightedFocalLoss(nn.Module):
    def forward(self, logits, targets):
        ce = F.cross_entropy(logits, targets,
                             weight=self.w, reduction="none")
        pt = torch.exp(-ce)
        return ((1 - pt) ** self.gamma * ce).mean()
```

The combination of class weights and focal loss is stronger than either alone. Class weights correct the base rate; focal loss corrects the difficulty distribution. For hateful memes, many borderline cases exist (e.g., sarcasm that an annotator might label differently), so focusing on hard examples is especially valuable.

5.4 Learning Rate Schedule

The notebook uses cosine annealing with linear warmup:

$$\mathrm{lr}(t) = \begin{cases} \mathrm{lr}_{\max} \cdot \frac{t}{T_{\text{warm}}} & \text{if } t < T_{\text{warm}} \\ \frac{\mathrm{lr}_{\max}}{2}\left(1 + \cos\left(\pi \cdot \frac{t - T_{\text{warm}}}{T_{\text{total}} - T_{\text{warm}}}\right)\right) & \text{otherwise} \end{cases}$$

This schedule prevents the model from overfitting in later epochs by gradually reducing the learning rate toward zero.

5.5 Training Loop Details

The training loop uses PyTorch's `GradScaler` for mixed-precision training:

1. Forward pass in `float16` (via `autocast`).
2. Compute focal loss.
3. Scale gradients and call `optimizer.step()`.
4. Update learning rate via scheduler.
5. Evaluate on dev set at the end of each epoch.
6. Save checkpoint if AUROC improves (early-stopping by checkpoint).

After each epoch, the loop prints:

```
Epoch 5/10  loss=0.312  val_auroc=0.702  val_f1=0.632  [42s]
```

5.6 Ablation Results

The four variants produce the following training curves and final dev-set metrics:

Variant	Best AUROC	Best F1
cross_attention	∼0.70	∼0.63
image_only	∼0.64	∼0.47
text_only	∼0.63	∼0.45
late_fusion	∼0.67	∼0.58

Table 5.1: Ablation study results on the dev set.

Key observations:

- **cross_attention** achieves the highest AUROC, confirming that cross-modal interaction matters.
- **image_only** and **text_only** perform similarly, reinforcing the finding from notebook 06 that neither modality dominates.
- **late_fusion** (averaging logits) improves over unimodal variants but falls short of cross-attention. This shows that *how* you fuse matters, not just *that* you fuse.

5.7 Training Curves Plot

The notebook generates a three-panel plot showing:

1. **Training loss** over epochs for all four variants (all decrease smoothly, confirming convergence).
2. **Validation AUROC** over epochs (cross-attention leads after epoch 2–3).
3. **Validation F1** over epochs (tracks AUROC trends).

The cross-attention variant's loss curve is the smoothest and converges fastest, indicating that the richer fusion mechanism provides a more informative gradient signal.

5.8 What Notebook 08 Teaches

This notebook teaches **controlled experimentation**. Instead of training only the best model and reporting its score, the notebook runs four variants under identical conditions (same data, same hyperparameters, same random seeds) and compares them.

The ablation proves two things:

1. Both modalities contribute (image-only ≈ text-only, but both below fusion).
2. Cross-attention fusion outperforms naive late fusion.

These are the two core claims of the project's architecture design. Without the ablation, they would be unsupported assertions.

Chapter 6

Notebook 09: AUROC Optimisation (Phase 2)

Notebook 08 established a baseline AUROC of ∼0.70 with the frozen-CLIP cross-attention model. Notebook 09 attempts to push this further by selectively unfreezing CLIP backbone layers and applying additional regularisation techniques.

6.1 High-Level Goal

The notebook implements a **phase 2 fine-tuning** strategy:

1. Load the best checkpoint from notebook 08 (warm start).
2. Unfreeze the last 2 layers of CLIP's vision and text encoders.
3. Train with differential learning rates: a small rate for backbone layers, a larger rate for the head.
4. Apply text augmentation and label smoothing as additional regularisation.

6.2 Why Phase 2 Is Separate

The two-phase training strategy is deliberate:

- **Phase 1** (notebook 08): train only the head with a frozen backbone. This learns a strong cross-attention fusion layer without risking catastrophic forgetting.
- **Phase 2** (notebook 09): unfreeze a small number of backbone layers and fine-tune with a very low learning rate. The warm-started head already knows how to use CLIP features, so the backbone adjustment is gentle.

Trying to do both at once—training a random head while also fine-tuning the backbone—would be unstable because the head's random gradients would corrupt the pretrained backbone.

6.3 Configuration Differences from Notebook 08

```
CFG = {
    'epochs'          : 20,
    'head_lr'         : 2e-4,
    'encoder_lr'      : 3e-6,
```

```
    'unfreeze_vision' : 2,   # last 2 vision transformer layers
    'unfreeze_text'   : 2,   # last 2 text transformer layers
    'label_smoothing' : 0.1,
    'text_aug_prob'   : 0.3,
    'dropout'         : 0.3,
    'focal_gamma'     : 2.0,
}
```

Key differences:

- **Differential LR**: the encoder learning rate (3×10^{-6}) is $\sim$67$\times$ smaller than the head learning rate (2×10^{-4}). This prevents catastrophic forgetting while still allowing the backbone to adapt slightly.
- **Label smoothing** $\epsilon = 0.1$: softens hard labels ($0 \to 0.05$, $1 \to 0.95$) to prevent the model from becoming overconfident, which improves calibration.
- **Text augmentation**: randomly perturbs training text with 30% probability (case swaps, punctuation insertion, word reordering) to improve robustness.
- **More epochs (20)**: fine-tuning with a small encoder LR requires more epochs to converge.

6.4 Text Augmentation Strategy

The `perturb_text` function applies one of three random perturbations:

1. **Case swap**: randomly change a word to uppercase or lowercase. This teaches the model that "HATE" and "hate" carry similar meaning.
2. **Punctuation insertion**: add !, ?, or ... to a random word. This simulates the noisy punctuation common in memes.
3. **Word swap**: swap two adjacent words. This teaches robustness to minor word-order variations.

Why these specific augmentations? Meme text is inherently noisy: it uses non-standard capitalisation, unconventional punctuation, and informal grammar. The augmentations simulate this noise distribution, effectively increasing the training set's diversity without adding new samples.

6.5 Focal Loss with Label Smoothing

Notebook 09 combines focal loss with label smoothing in a single loss function:

```
class FocalLossWithSmoothing(nn.Module):
    def forward(self, logits, targets):
        # Focal weight from hard targets
        ce_hard = F.cross_entropy(logits, targets,
                                  reduction='none')
        focal_weight = (1 - torch.exp(-ce_hard)) ** self.gamma

        # Label-smoothed CE
        log_probs = F.log_softmax(logits, dim=-1)
        smooth_tgts = torch.full_like(log_probs,
                      self.smoothing / (n - 1))
        smooth_tgts.scatter_(1, targets.unsqueeze(1),
```

```
                                1.0 - self.smoothing)
        smooth_ce = -(smooth_tgts * log_probs).sum(dim=-1)

        return (focal_weight * smooth_ce).mean()
```

Interaction between focal and smoothing:

- The focal weight is computed from hard targets (no smoothing), so it still focuses on genuinely hard examples.
- The CE loss itself uses smoothed targets, so the model is never trained to output extreme probabilities.
- Together, they create a loss that is forgiving of ambiguous labels (smoothing) while still pushing the model on challenging cases (focal).

6.6 Selective Backbone Unfreezing

The notebook unfreezes the last 2 layers of both CLIP encoders. The implementation identifies specific parameter groups:

- `vision_model.encoder.layers[-2:]` — the last two transformer blocks of the vision encoder.
- `text_model.encoder.layers[-2:]` — the last two transformer blocks of the text encoder.
- All other CLIP parameters remain frozen.

Why only the last 2 layers? In deep networks, earlier layers learn general features (edges, textures, basic syntax) while later layers learn task-specific features. Unfreezing only the last layers allows the model to adapt its high-level representations while preserving the general features that CLIP learned from 400M pairs.

6.7 Training Output and Results

The training loop produces epoch-by-epoch metrics, compared against the notebook 08 reference AUROC:

```
Reference NB-08 AUROC : 0.7017
Phase 2 best AUROC    : (result)
Delta                 : (improvement)
```

The three-panel training curve plot shows:

1. **Train loss**: decreasing with the characteristic shape of cosine annealing (flattening at the end of cycles).
2. **Validation AUROC**: plotted against the NB-08 baseline as a dashed horizontal line. The goal is to exceed this line.
3. **Validation F1 and Accuracy**: tracked to ensure that AUROC improvements correspond to actual classification improvements.

6.8 Why AUROC Is the Optimisation Target

AUROC is preferred over accuracy or F1 because:

$$\text{AUROC} = P(\hat{s}(\mathbf{x}^+) > \hat{s}(\mathbf{x}^-))$$

where $\mathbf{x}^+$ is a random positive (hateful) sample and $\mathbf{x}^-$ is a random negative sample. AUROC directly measures the model's ability to rank hateful memes above non-hateful ones, regardless of the classification threshold.

In a content moderation system, the threshold would be tuned based on the desired false positive rate. Optimising AUROC ensures that the ranking quality is as high as possible, giving downstream threshold tuning the best starting material.

6.9 What Notebook 09 Teaches

This notebook teaches **surgical fine-tuning**. The key insights are:

1. **Two-phase training** is safer than one-phase for transfer learning: first learn a good head, then gently adapt the backbone.
2. **Differential learning rates** are essential when combining pretrained and randomly-initialised parameters.
3. **Text augmentation** for short, noisy text (like memes) should simulate the actual noise distribution, not generic NLP perturbations.
4. **Label smoothing + focal loss** complement each other: smoothing prevents overconfidence, focal prevents under-attention to hard cases.

Chapter 7

Cross-Notebook Patterns and Design Lessons

7.1 Progressive Complexity

The five notebooks follow a strict progression:

1. **Preprocessing** (05): transform raw data into model-ready format.
2. **Baselines** (06): establish what simple models can do.
3. **Architecture** (07): define the target model.
4. **Training** (08): train and ablate.
5. **Optimisation** (09): push performance further.

No notebook skips a level. The project does not jump from raw data to a complex model. Each step justifies the next.

7.2 Each Notebook Is Self-Contained

Every notebook re-discovers the data directory, re-defines necessary classes inline, and does not import from `src/` modules. This makes each notebook independently runnable on Kaggle or locally.

The trade-off is code duplication. The `MemeDataset` class appears in notebooks 05, 08, and 09 with minor variations. This is a deliberate choice: for a research project, reproducibility of individual experiments is more important than DRY (Don't Repeat Yourself) principles.

7.3 AUROC as the North Star Metric

Across all five notebooks, AUROC is the primary metric. Accuracy and F1 are tracked but never used for decision-making. This consistency ensures that model selection and hyperparameter tuning are not confounded by threshold-dependent metrics.

7.4 Ablation-Driven Design

The project does not assert that cross-attention is better than late fusion. It *proves* it through controlled ablation (notebook 08). Similarly, the progression from TF-IDF to CLIP fusion in

notebook 06 is evidence-based, not opinion-based.

This experimental rigour is unusual in applied ML projects and reflects a research-grade methodology.

7.5 Regularisation Philosophy

Across notebooks 08 and 09, the project applies five complementary regularisation techniques:

1. **Dropout** (0.3): random feature zeroing in the MLP head.
2. **Weight decay** (0.01): L2 penalty on all trainable parameters.
3. **Focal loss**: implicit regularisation through hard-example focusing.
4. **Label smoothing** (0.1): prevents extreme confidence.
5. **Text augmentation**: data-level regularisation.

Each addresses a different failure mode. Together, they create a robust training pipeline for a small-data regime.

Chapter 8

Concise Notebook-by-Notebook Summary

Notebook 05 (Preprocessing)
Cleans text, defines CLIP-compatible transforms, builds the `MemeDataset` class, computes class-balance weights, and saves three cleaned JSONL files. Every downstream notebook depends on its outputs.

Notebook 06 (Baselines)
Trains four baselines: TF-IDF + LR (AUROC = 0.594), RoBERTa (0.645), CLIP image-only (0.647), and CLIP concat fusion (0.700). Demonstrates that multimodal fusion is necessary and that neither modality alone is sufficient.

Notebook 07 (Architecture)
Defines the full model: frozen CLIP encoder (151M params) + cross-attention fusion + MLP head (2.4M trainable params). Includes forward-pass smoke test and attention visualisation.

Notebook 08 (Training)
Trains four model variants with focal loss, cosine scheduling, and mixed precision. Cross-attention achieves $\sim$0.70 AUROC, outperforming late fusion ($\sim$0.67) and unimodal variants ($\sim$0.63–0.64).

Notebook 09 (AUROC Optimisation)
Phase 2 fine-tuning: unfreezes last 2 CLIP layers with differential LR (3×10^{-6} encoder, 2×10^{-4} head), adds label smoothing and text augmentation. Targets AUROC improvement over the NB-08 baseline.

Chapter 9

Final Interpretation

Notebooks 05–09 transform the project from an analytical exercise into a functional multimodal classifier. The design is notable for its discipline:

- Preprocessing is isolated and reproducible (notebook 05).
- Claims about multimodal superiority are backed by a four-baseline ladder (notebook 06).
- The architecture is documented as a standalone artefact with parameter accounting and smoke tests (notebook 07).
- Training includes rigorous ablation across four structural variants (notebook 08).
- Optimisation uses principled techniques—differential LR, label smoothing, focal loss—each with explicit rationale (notebook 09).

The resulting model achieves an AUROC in the 0.70+ range on Meta's Hateful Memes dev set, which is competitive for a ViT-B/32 backbone without ensembling or external data. For context, the original Meta challenge baseline (ViLBERT fine-tuned) achieved AUROC = 0.714.

The next set of notebooks (10–14) will evaluate this model on the test set, explain its predictions, test it with counterfactuals, analyse errors, and integrate human-in-the-loop review.

Appendix A

Key Numbers Used in This Report

The following values were extracted directly from the notebook outputs while preparing this guide:

- Split sizes: train 8,500; dev 500; test 1,000.
- Train balance: 5,450 non-hateful, 3,050 hateful.
- Dev balance: 250 non-hateful, 250 hateful (balanced).
- Class weights: non-hateful = 0.780, hateful = 1.393.
- CLIP model: `openai/clip-vit-base-patch32`.
- Image size: 224×224 pixels.
- CLIP embedding dimension: 512.
- Fused embedding dimension: 1024.
- Total model parameters: 153,676,163.
- Frozen parameters (CLIP): 151,277,313 (98.4%).
- Trainable parameters: 2,398,850 (1.6%).
- Cross-attention heads: 4.
- MLP architecture: $1024 \rightarrow 256 \rightarrow 128 \rightarrow 2$.
- Focal gamma: 2.0.
- Label smoothing epsilon: 0.1.
- Baseline AUROC ladder: $0.594 \rightarrow 0.645 \rightarrow 0.647 \rightarrow 0.700$.
- Phase 1 best AUROC: $\sim$0.70.
- Phase 2 encoder LR: 3×10^{-6}.
- Phase 2 head LR: 2×10^{-4}.
- Unfrozen layers: 2 (vision) + 2 (text).

Appendix B

Suggested Reading Order for a Beginner

If you are new to this codebase, read the notebooks in this order:

1. **Notebook 05**: understand how raw data becomes model-ready data. Pay attention to the text-cleaning design choices and the CLIP-specific normalisation constants.
2. **Notebook 06**: study the baseline table carefully. The AUROC progression from 0.594 to 0.700 is the empirical motivation for every subsequent design decision.
3. **Notebook 07**: read the architecture code line by line. Understand why cross-attention is used instead of concatenation. Run the forward-pass test mentally to verify your understanding.
4. **Notebook 08**: focus on the ablation results. The comparison between cross-attention, late fusion, and unimodal variants is the project's strongest empirical claim.
5. **Notebook 09**: understand the two-phase training rationale. The differential learning rate and selective unfreezing are the most technically subtle parts of the pipeline.

After these five notebooks, proceed to notebooks 10–14 for evaluation, explainability, counterfactual analysis, error analysis, and human-in-the-loop review.